

29. Compter à l'envers, et le piège du zéro

Un octet n'est jamais plus petit que zéro : « `i >= 0` » est toujours vrai.

1. Ce qu'il faut comprendre

Compter à l'envers change les trois morceaux : partir **du haut**, continuer **tant qu'on est au-dessus** du bas, et **retirer** un à chaque tour : `for (uint8_t i = 9; i > 0; i--)` donne 9, 8 ... 1.

Et zéro ? Le réflexe est d'écrire `i >= 0`. **C'est une boucle infinie**. Un `uint8_t` ne descend jamais sous zéro : après 0, `i--` donne **255**, qui est `>= 0`, et l'on repart pour 256 tours... à l'infini. Le compilateur ne dit rien — la ligne est parfaitement correcte en C++.

La bonne façon : compter de 10 à 1, et **se servir de** `i - 1`. Quand `i` vaut 1, on travaille sur 0, puis `i` devient 0 et la condition `i > 0` arrête tout proprement.

`tours` compte les passages : **10** exactement. Compter les tours est la meilleure façon de vérifier une boucle qu'on n'est pas sûr d'avoir bien écrite.

À toi : avec une boucle `for` qui monte, affiche 9 à 0 quand même (indice : `9 - i`).

Les fonctions de cette leçon — et le tuto de chacune

- `#include <ecran>` → [27.1. La fonction `ecran\(\)` — éteindre et rallumer l'écran](#)
- `#include <texte>` → [0.0.2. La fonction `texte\(\)` — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction `nombre\(\)` — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **753 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage — la mise en route de la console
main()         171 o ← démarrage — ta fonction
VBlank         8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage  28 o ← image() — attend l'image suivante
AttendreVBlank 16 o ← écran() — attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
AdresseCase    10 o ← nombre() — trouve l'adresse d'une case de l'écran
EcrireTexte    10 o ← texte() — écrit un texte, lettre par lettre
EcrireNombre   33 o ← nombre() — écrit un nombre en chiffres
CopierTuiles   76 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage — vide les deux cartes de l'écran
Tuiles         304 o ← CopierTuiles — 19 lettres sur 43
"DE 9 A 1"     8 o ← texte() — un texte gravé
"DE 9 A 0"     8 o ← texte() — un texte gravé
"TOURS:"       6 o ← texte() — un texte gravé
```

Lettres gravées : ADEORSTU0123456789:

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert,

Notes

Fonctions du langage appelées :

nombre()	× 3	64 o sur place, dans main()	+ AdresseCase (10 o), EcrireNombre (33 o)
texte()	× 3	33 o sur place, dans main()	+ EcrireTexte (10 o)
ecran()	× 2	21 o sur place, dans main()	+ AttendreVBlank (16 o)
image()	× 1	3 o sur place, dans main()	+ AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : compte à l'envers avec for : de 9 à 1, puis de 9 à 0.
//
// Le piège : un uint8_t ne descend jamais sous 0 (après 0, il revient à 255).
// Donc « i >= 0 » est TOUJOURS vrai, et une boucle qui s'arrête sur « i >= 0 »
// ne s'arrête jamais.

#include <ecran>    // éteint ou rallume l'écran
#include <texte>    // écrit un texte à l'écran
#include <nombre>   // écrit un nombre en chiffres
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    écran(0);                               // Tout le dessin se fait écran éteint (plus rapide).

    texte(1, 1, "DE 9 A 1");
    for (uint8_t i = 9; i > 0; i--) {        // i part de 9 ; on continue tant que i > 0 ;
                                                // « i-- » retire 1 à chaque tour : 9, 8, ... 1.
        nombre((9 - i) * 2, 3, i, 1);        // La colonne : (9 - i) * 2.
                                                // i = 9 → colonne 0 ; i = 8 → colonne 2 ; ...
    }

    // for (uint8_t i = 9; i >= 0; i--) ne s'arrête JAMAIS
    // (0 - 1 donne 255, qui est encore >= 0).

    // La bonne façon d'aller jusqu'à 0 : compter de 10 à 1, et afficher i - 1.
    texte(1, 7, "DE 9 A 0");
    uint8_t tours = 0;                       // Combien de tours la boucle a faits.
    for (uint8_t i = 10; i > 0; i--) {        // i = 10, 9, ... 1 : dix tours.
        nombre((10 - i) * 2, 9, i - 1, 1);    // On affiche i - 1 : 9, 8, ... 0.
        tours++;                             // Un tour de plus.
    }

    texte(1, 13, "TOURS:");
    nombre(8, 13, tours);                   // Affiche 010 : la boucle a bien fini.

    écran(1);                               // On rallume l'écran : tout apparaît d'un coup.

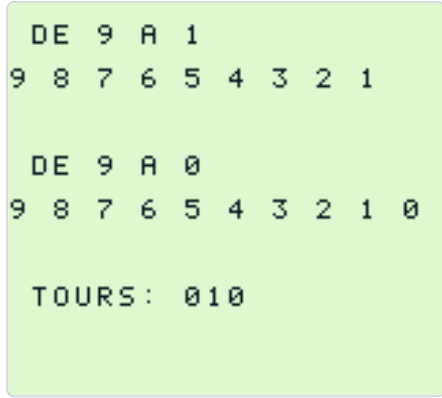
    while (true) {                           // La boucle du jeu.
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

9 8 7 ... 1 sur la première ligne, 9 8 7 ... 0 sur la seconde, et 010 tours.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ de 9 à 1
- ✓ de 9 à 0, sans boucle infinie
- ✓ exactement dix tours

Ensuite : **30. Une boucle dans une boucle : la grille** — chapitre 5 : Les boucles : répéter. · gameboy3 — écrire une cartouche Game Boy en C++